

```
In [1]: import pandas as pd
import numpy as np
```

```
In [3]: df=pd.read_excel(r"C:\Users\HP\Downloads\supply_chain_sales.xlsx")
df
```

Out[3]:

	Date	Product	Sales	Price	Promotion	Holiday	Temperature
0	2024-01-01	Product A	119	490	0	0	20
1	2024-01-02	Product A	125	500	0	0	20
2	2024-01-03	Product A	139	500	1	0	20
3	2024-01-04	Product A	126	500	0	0	21
4	2024-01-05	Product A	113	490	0	0	25
...	...	...	...	...	...	...	...
360	2024-12-26	Product A	105	510	0	0	20
361	2024-12-27	Product A	115	510	0	0	20
362	2024-12-28	Product A	116	500	0	0	22
363	2024-12-29	Product A	117	510	0	1	21
364	2024-12-30	Product A	108	490	0	0	20

365 rows × 7 columns

```
In [5]: df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 365 entries, 0 to 364
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        365 non-null   object
1   Product     365 non-null   object
2   Sales       365 non-null   int64
3   Price       365 non-null   int64
4   Promotion   365 non-null   int64
5   Holiday     365 non-null   int64
6   Temperature 365 non-null   int64
dtypes: int64(5), object(2)
memory usage: 20.1+ KB
```

```
In [7]: df.dtypes
```

```
Out[7]: Date      object
        Product   object
        Sales     int64
        Price     int64
        Promotion  int64
        Holiday   int64
        Temperature int64
        dtype: object
```

```
In [9]: df["Date"] = pd.to_datetime(df["Date"])
        df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 365 entries, 0 to 364
Data columns (total 7 columns):
#   Column      Non-Null Count  Dtype
---  -
0   Date        365 non-null   datetime64[ns]
1   Product     365 non-null   object
2   Sales       365 non-null   int64
3   Price       365 non-null   int64
4   Promotion   365 non-null   int64
5   Holiday     365 non-null   int64
6   Temperature 365 non-null   int64
dtypes: datetime64[ns](1), int64(5), object(1)
memory usage: 20.1+ KB
```

```
In [11]: df["Day"] = df["Date"].dt.day
         df["Month"] = df["Date"].dt.month
         df["Year"] = df["Date"].dt.year

         df.head()
```

Out[11]:

	Date	Product	Sales	Price	Promotion	Holiday	Temperature	Day	Month	Year
0	2024-01-01	Product A	119	490	0	0	20	1	1	2024
1	2024-01-02	Product A	125	500	0	0	20	2	1	2024
2	2024-01-03	Product A	139	500	1	0	20	3	1	2024
3	2024-01-04	Product A	126	500	0	0	21	4	1	2024
4	2024-01-05	Product A	113	490	0	0	25	5	1	2024

```
In [13]: df.isnull().sum()
```

```
Out[13]: Date          0
        Product        0
        Sales          0
        Price          0
        Promotion      0
        Holiday        0
        Temperature    0
        Day            0
        Month          0
        Year           0
        dtype: int64
```

```
In [15]: df.duplicated
```

```
Out[15]: <bound method DataFrame.duplicated of
motion Holiday Temperature Day \
0  2024-01-01 Product A    119  490      0      0      20      1
1  2024-01-02 Product A    125  500      0      0      20      2
2  2024-01-03 Product A    139  500      1      0      20      3
3  2024-01-04 Product A    126  500      0      0      21      4
4  2024-01-05 Product A    113  490      0      0      25      5
..    ...    ...    ...    ...    ...    ...    ...    ...
360 2024-12-26 Product A    105  510      0      0      20     26
361 2024-12-27 Product A    115  510      0      0      20     27
362 2024-12-28 Product A    116  500      0      0      22     28
363 2024-12-29 Product A    117  510      0      1      21     29
364 2024-12-30 Product A    108  490      0      0      20     30

        Month  Year
0           1  2024
1           1  2024
2           1  2024
3           1  2024
4           1  2024
..    ...    ...
360      12  2024
361      12  2024
362      12  2024
363      12  2024
364      12  2024

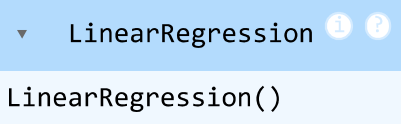
[365 rows x 10 columns]>
```

```
In [17]: from sklearn.model_selection import train_test_split

x = df[['Day', 'Month', 'Year']]
y = df['Sales']

x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.2, random_state=42
)
```

```
In [19]: from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
```

Out[19]:  LinearRegression()

In [21]: `predctions=model.predict(x_test)`

In [23]: `from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score`
`mae=mean_absolute_error(y_test,predctions)`
`print("The Absolute Error:",round(mae,2))`

The Absolute Error: 10.52

In [25]: `future=pd.DataFrame({`
`'Day':[11,12,13],`
`'Month':[1,1,1],`
`'Year':[2024,2024,2024]`
`})`

In [27]: `future_prediction=model.predict(future)`
`print("\nFuture demand Prediction")`
`for day,demand in zip(future['Day'],future_prediction):`
`print(f"Day {day}:{round(demand)}units")`

Future demand Prediction
 Day 11:130units
 Day 12:130units
 Day 13:130units

In [29]: `future_pred=model.predict(future)`
`future_pred`

Out[29]: `array([130.21491095, 130.1007789 , 129.98664685])`

In [31]: `#demand optimizes`
`current_inventory=120`
`predicted_demand=185`
`safety_stock=30`
`reorder_qunatity=predicted_demand+safety_stock-current_inventory`
`if reorder_qunatity>0:`
`print("order qunatity",reorder_qunatity,"units")`
`else:`
`print("inventory is sufficient")`

order qunatity 95 units

In [33]: `current_inventory=250`
`predicted_demand=185`
`safety_stock=30`
`reorder_qunatity=predicted_demand+safety_stock-current_inventory`
`if reorder_qunatity>0:`
`print("order qunatity",reorder_qunatity,"units")`

```
else:  
    print("inventory is sufficient")
```

inventory is sufficient